Robotique
CRC
Robotics

CRC

# PROGRAMMING COMPETITION




# PROBLEM BOOKLET BLOCK B

## A FEW NOTES

- The complete rules are in section 4 of the rulebook.
- You have until **14h59** to submit your solutions.
- Feel free to ask organizers questions and discuss the problem with your team members. It is not an exam!
- **We are giving you quick and easy-to-use template files. Please use them!**

## USING THE TEMPLATE FILE

- All the files contain a function called solve() where you have to put your code. **DO NOT CHANGE THE NAME OF THAT FUNCTION!**
- To run the tests you will have to open a terminal (we strongly recommend to use vscode) and run the command `pytest`. If the command is ran in the folder containing all the problems, all tests are going to be ran
- To run a specific test, run it in the folder containing the test you want. You can do the same for sections, just navigate in the folder before executing the command `pytest`

## STRUCTURE

Every problem contains a small introduction like this about the basics of the problem and what is required to solve it. Points distribution is also given here for the preliminary problems.

### Input and output specification:

In these two sections, we specify what the inputs will be and what form they will take, and we also say what outputs are required for the code to produce and in what format they shall be.

### Sample input and output:

In these two sections, you will find a sample input (that often has multiple entries itself) in the sample input and what your program should produce for such an input in the sample output.

### First output explanation:

Sometimes, the problem might still be hard to understand after those sections, which is why there will also be a usually brief explanation of the logic that was used to reach the first output from the first input.

# Table des matières


**CHECKMATE (CM) 3**

CM1: Queens everywhere (50 points) 3

CM2: Horse`s dance (70 points) 4

**WONDERS OF THE WORLD (WW) 6**

WW1  Great pyramid of Giza (35 points) 6

WW2: Hanging gardens of Babylon (15 points) 8

WW3 - Zeus statue (50 points) 10

WW4: Temple of Arthemis (30 points) 12

WW5: Mausoleum at Halicarnassus (15 points) 14

WW6: Colossus of Rhodes (40 points) 15

WW7: Lighthouse of Alexandria (85 points) 1

**MM Mastermind 1**

MM: Master of the mind (100 points) 1

# CHECKMATE (CM)

## CM1: Queens everywhere (50 points)

You'll need to place queens on a chessboard equal to the size of the side of the square. None of the checkers may see another queens. Queens in a chess set can move in a line of as many squares as possible, in a row or diagonally, and no queen can see another queen. In short, if you have a 4x4 grid, you'll need to fit 4 queens that can't see each other.

### Input specification:

As input you will receive:

- ***size***: length of one of the sides of the board and also the number of checkers you'll need to put on it

### Output specification:

As output, you must provide:

- an array of 2 int arrays representing the positions of the queens on the board

### Input example

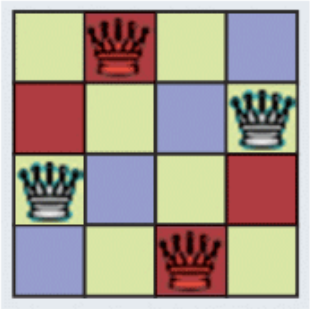
```
4
8
9
```

### Output example:

```
There are several possible answers for the same input, but
here's a valid output for example 1
[[0, 1], [1, 3], [2, 0], [3, 2]]
```

### Output explanation:

Note that each queen has no other queens on the same column, row or diagonal. None of them can be seen.



## CM2: Horse`s dance (70 points)

A knight (from chess, of course) moves like a chess knight on an 8 by 8 chess board where the squares are numbered from 0 to 63 in an ascending serpentine pattern:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 63 | 62 | 61 | 60 | 59 | 58 | 57 | 56 |
| 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 |
| 47 | 46 | 45 | 44 | 43 | 42 | 41 | 40 |
| 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 |
| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
| 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

When the knight moves on this chessboard, he accumulates Kryptik pieces corresponding to the number of the square on which he has landed. However, as the knight fights wealth and is completely anti-materialistic, he wants to accumulate as few Kryptik pieces as possible. Your knight will "land" on a starting square (meaning it counts towards the pieces total) and must reach the finishing square with as few coins as possible. You'll need to give the total number of pieces accumulated for the optimal trajectory.

### Input specification:

As input you will receive:

- ***start***: an integer corresponding to the rider's initial square number
- ***end***: an integer corresponding to the number of the rider's arrival square

## Output specification:

As output you will need to provide:

- an integer giving the number of Kryptik pieces accumulated for an optimal trajectory

## Input example:

```
start: 35          end: 34
start: 23          end: 37
start: 0           end: 63
start: 50          end: 55
```

## Output example:

```
114
93
167
182
```

## Output explanation:

We won't compare all the possible paths, as there are so many. There are two optimal paths from 35 to 34. The first passes through squares 35-26-19-34 and the second through 35-18-27-34. The sum of any of these paths is 114.

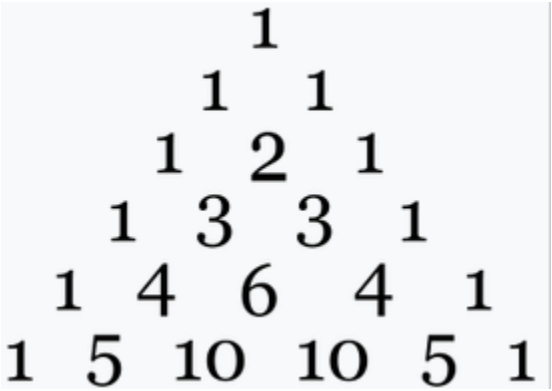
Footnote:

Did you know that you can place up to 32 riders on a chessboard without any of them being able to see each other?

# WONDERS OF THE WORLD (WW)

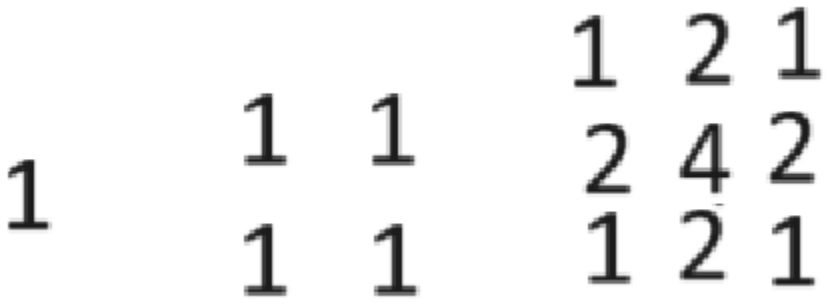
## WW1  Great pyramid of Giza (35 points)

Pascal's triangle is a particular number construction where we add the numbers of one stage in pairs to find the next stage, starting with a simple 1. Here are its first six levels:




The logic is that each number "transmits" to the two immediately diagonally below. It's also as if there were invisible 0's surrounding each floor, explaining why each floor is always framed by 1's. Let's look at the passage from floor 5 to floor 6, for example. 0+1 = 1, 1+4=5, 4+6=10, 6+4=10, 4+1=5 and 1+0=1, which corresponds well to the sixth floor.

**We'll ask you to perform the same principle, but on a three-dimensional pyramid. In this situation, each number on the floor above is transmitted to the 4 numbers immediately diagonally opposite:**




Here are the first three floors of such a pyramid. Moving on from floor 2 to floor 3, and considering the "invisible" 0's all around, we get 0+0+0+1=1, 0+0+1+1=2, 0+0+1+0=1, 0+1+0+1=2, 1+1+1+1=4, 1+0+1+0=2, 0+1+0+0=1, 1+1+0+0=2 and 1+0+0+0=1, which

completes the third floor. The principle is the same at every level of the pyramid. Naturally, this means that each *n-level* is always an *n-by-n* square, with 1's in all four corners

## Input specification:

As input you will receive:

- ***level***: an int for the level to output

## Output specification:

You will output:

- A 2D array of the level of the pyramid

## Input example:

```
3
2
5
7
```

## Output example:

```
[[1,2,1],[2,4,2],[1,2,1]]
[[1,1],[1,1]]
[[1,4,6,4,1],[4,16,24,16,4],[6,24,36,24,6],[4,16,24,16,4],[1,4,6
,4,1]]
[[1,6,15,20,15,6,1],[6,36,90,120,90,36,6],
[15,90,225,300,225,90,15],[20,120,300,400,300,120,20],
[15,90,225,300,225,90,15],[6,36,90,120,90,36,6],
[1,6,15,20,15,6,1]]
```

## Output explanation:

See the problem description to see how we moved from the second to the third floor. To move from the first to the second floor, simply transmit the initial 1 to its 4 neighbors below.

## Footnote:

Find out more about Pascal's triangle and discover many interesting applications in science.

## WW2: Hanging gardens of Babylon (15 points)

The Hanging Gardens of Babylon contain an enormous diversity of coexisting plants. To mark the visit of a king from a faraway kingdom, Babylon's gardeners decided to decorate the main entrance to the gardens with a sample of the best plants the king would see during his visit. However, this task poses a slight problem: not all species of flowering plants have the same flowering time. The decorating team will therefore have to take great care to ensure that all the plants bloom at the right time.
We'll need a calendar that includes when each type should be planted. So the calendar starts twelve months before the visit, element 0 must contain the plants that are due to flower in twelve months, element 1 takes 11 months to flower and so on.
If ever two plants have the same time before flowering, they must be in alphabetical order in the calendar, with a space between them in the display.

### Input specifications:

As input you will receive:

- ***flowers***: an array containing arrays of two strings, one for the plant name and the other for the flowering time

### Output specifications:

As output, you'll need to provide:

- an array containing one string per month of plants needing to be planted

### Input example:

```
[[‘rose’, ‘8’],
[‘tulip’, ‘6’],
[‘petunia’, ‘4’],
[‘sunflower’, ‘5’],
[‘orchid’, ‘4’],
[‘lily’, ‘11’],
[‘iris’, ‘8’]]

[[‘sunflower’, ‘5’],
[‘petunia’, ‘4’],
[‘lily’, ‘11’],
[‘geranium’, ‘7’],
[‘daffodil’, ‘12’],
```

```
['lavender', '2']]

[['lily', '11'],
['geranium', '7'],
['daffodil', '12'],
['iris', '8'],
['lavender', '2']]
```

Output example:

```
['', 'lily', '', '', 'iris rose', '', 'tulip', 'sunflower',
'orchid petunia', '', '', '']

['daffodil', 'lily', '', '', '', 'geranium', '', 'sunflower',
'petunia', '', 'lavender', '']

['daffodil', 'lily', '', '', 'iris', 'geranium', '', '', '',
'', 'lavender', '']
```

Output explanation:

For the first example, there are no flowers requiring 12 months, only lilies for 11 months. Nothing for 10 and 9 months. For 8 months we have iris and rose separated by a space and arranged alphabetically. There are no flowers for 7 months in advance, then there are tulips 6 months in advance, sunflowers 5 months in advance. Finally orchids and petunias 4 months in advance and no flowers for the last 3 months.

# WW3 - Zeus statue (50 points)

On a trip with her school to the city of Olympia, Claire Éclairé follows her entrepreneurial spirit and asks her friends how much money they could generate if they "recycled" the materials from the statue of Zeus. Unable to sleep at night with the question running through their heads, the group decides to go and collect data to put an end to their anguish.

They start by going to the statue to measure the dimensions of the rough from which the statue was made (if this is an anachronism, they have a time machine). Before he began sculpting the statue, Phidias (the artist who created this wonder of the world) had only a large square-based prism in his workshop. Armed with a measuring tape, our group of students managed to measure the length of one of the sides of the base of this block. They then get out their mass spectrometry equipment to determine the percentage of ivory and gold present in the statue (% ivory + % gold = 100%). The group then heads to the library to do some research. They first find the density of ivory (ρ/roh) with a simple internet search.

They then dive into dust-covered books to read the artist's original notes. They find the mass of the square prism used (raw) and the percentage of volume removed in the creation of this enormous work of art. Satisfied with the information they've gathered, the girls return to their hotel room to prepare the PowerPoint they want to present to potential investors. Their plan is to "reuse" the gold from the statue to make pure gold Christmas ornaments. They will melt the gold to cast balls of radius '**r**', which they will then sell at selling price **p**. They will, of course, carry out quality control to ensure that only complete balls are sold.
If they were to sell incomplete balls, their public image would be ruined before they could make a name for themselves in the field of luxury tree decorations. On the last slide of their audio-visual presentation, they show the estimated profit their project will generate if they "forget" to pay their labor. Please give us this profit as an output.

order of operation to create the statue:
-raw
-remove material -> finished ivory portion
-apply gold
-Voila!

## Input specification:

As input you will receive:

- ***l:*** *side length*
- ***i*** *:percentage of ivory in the statue*
- ***rohI:*** density ivory
- ***rohO:*** *density gold*
- ***m:*** *mass of raw material*
- ***e:*** *percentage of volume removed from the raw material to make the statue*
- ***r:*** *radius of spheres to be made*
- ***p:*** *selling price of a sphere*

## Output specification:

You'll need to provide

- a float: the amount raised after selling all the spheres

## Input example:

```
l: 6, i: 7, rohI: 8, rohO: 8.5, m: 9, e: 10, r: 0.5, p: 3
l: 40, i: 16, rohI: 8, rohO: 10, m: 200, e: 80, r: 1, p: 8
l: 33, i: 66, rohI: 0.9, rohO: 17, m: 666, e: 50, r: 3, p: 1000
l: 24, i: 90, rohI: 18, rohO: 1, m: 10, e: 60, r: 0.2, p: 3090
```

## Output example:

```
6
160
0
58710
```

## Output explanation:

Refer yourself to the problem description for the explanation. Part of the problem is to figure out the logic yourself!

## WW4: Temple of Arthemis (30 points)

Don’t ask me why but you have found yourself in the middle of a laser tag game. You are surrouThe Temple of Artemis contains many columns that fell during its destruction. As an archaeologist, you'd like to calculate the number of columns needed to support the roof of the temple according to certain dimensions. All around the building are rows of columns, except for the front façade, which contains two rows of columns. You'll receive the length and width of the building in column numbers.



### Input specification:

As input you will receive:

- ***length*:** an integer representing the length of the building in terms of columns
- ***width*:** an integer representing the width of the building in terms of columns

### Output specification:

For output, you will need to supply:

- an integer being the number of columns in the temple

Input example:

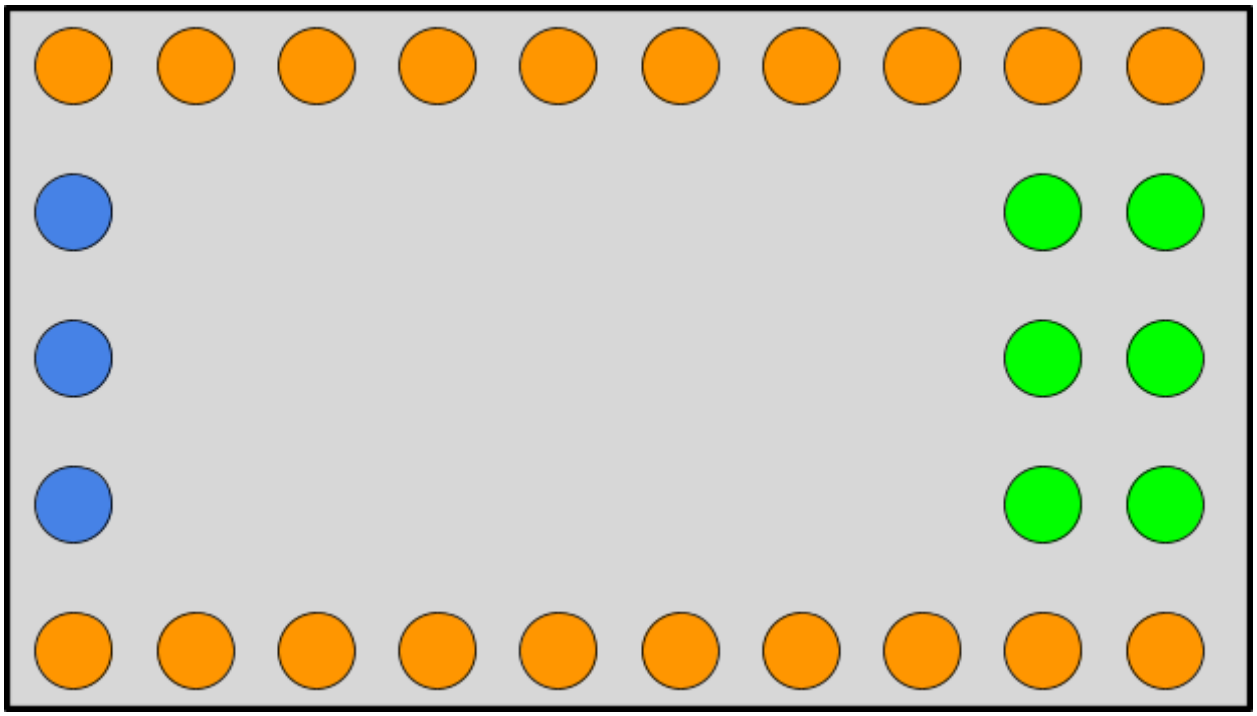
```
length: 10 width: 5
length: 20 width: 10
length: 11 width: 7
```

Output example:

```
29
64
37
```

Output explanation:

For a temple with a length of 10 columns and a width of 5 columns, we have a total of 29 columns. Let's start by separating the problem by looking at different sections of columns. First, on the side of the temple we have two rows of 10 columns, making 20 columns (orange). Then, at the back, we have 5 columns. However, 2 of these 5 columns have already been counted in the columns on the temple side, so there are only 3 (blue). In front of the temple we have 2 rows of 5 columns, so 10 columns, but don't forget that 4 of them have already been counted in the calculation of the side columns. This gives us 6 columns at the front (green). So we have 20 columns on the side + 3 more behind the temple + 6 more in front of the temple. 20 + 3 + 6 = 29



# WW5: Mausoleum at Halicarnassus (15 points)

During archaeological research on the mausoleum, you try to discover the age of the bodies buried in the mausoleum's tombs. To determine their age, you decide to use carbon dating.

To do this, you look at the current concentration of carbon 14, which you compare with the initial concentration of carbon 14 using the following formula. This will tell you how old the tombs in the mausoleum are. As the concentration values are low, they will be given to you using scientific notation. So, the value and its power of 10 by which to multiply it.

time =1ln(C0Ct)

Where C0=10-12 and =0.00012

Once you've found the time, as it's now 2024, all you have to do is subtract to find the year the tomb was created.

**Input specifications:**

As input you will receive:

v: a float representing the value of Ct before the exponent

ex: an int of the exponent applied to v to obtain Ct

**Output specifications:**

As output, you must provide:

an integer of the rounded value of the creation date of the grave

**Input example:**

v: 8.32 ex: -12

v: 8.26 ex: -12

v: 0.7999 ex: -11

**Output example:**

491

431

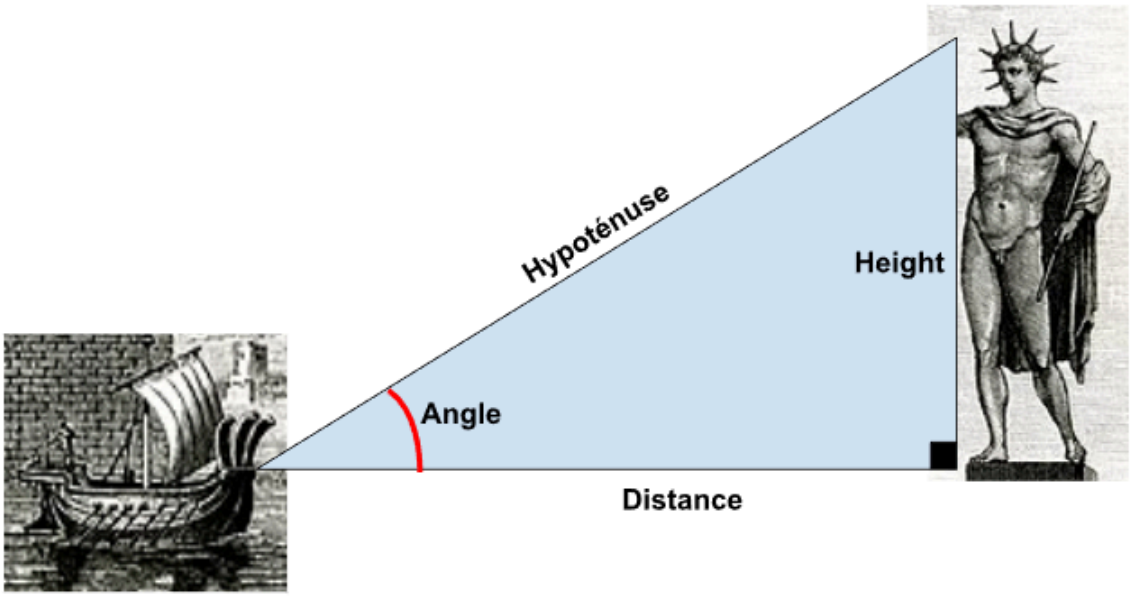
163

**Output explanation:**

For the first example, we take Ct=8.32 10-12 . We can calculate a time value equivalent to 1532.69031801. The result can be rounded off to give a number of years of 1533, and then subtracted from 2024 to give the year 491.

## WW6: Colossus of Rhodes (40 points)

You're on a boat in ancient Greece, and you can see the prestigious city of Rhodes in the distance. At the entrance to the city stands the huge statue of Helios, the god of the sun. This statue is often referred to as the Colossus of Rhodes!
Using your keen sense of observation and a sextant, you try to estimate the height of the statue. You receive an approximate distance in meters and the approximate angle between the statue's feet and head from your ship. You'll need to give the smallest possible height, as well as the greatest possible height of the statue.
You'll need to use the properties of right-angled triangles to find the height from the distance and angle measurements.




Here's a reminder of some useful trigonometry formulas. Given an angle and the distance from the statue's base, we can find the hypotenuse.

$$cos(Angle) = \frac{distance}{hypoténuse} \Rightarrow hypoténuse = \frac{distance}{cos(Angle)}$$

Now that we have the distance and the hypotenuse, we can apply the Pythagorean formula to obtain the height of the statue.

$$(hypoténuse)^2 = (distance)^2 + (height)^2$$

$$height = \sqrt{(hypoténuse)^2 - (distance)^2}$$

## Input specifications:

As input you will receive:

- ***distance***: an integer of the number of meters of distance
- ***distance_uncertainty*****: a float of the percentage of possible error**
- ***angle***: an integer (angle from 0 to 365 degrees)
- ***angle_uncertainty***: an integer of the number of degrees of uncertainty (from 0 to 365 degrees)

## Output specifications:

As output, you'll need

- an array containing two floats (3 digits after the decimal point) representing the upper and lower limits of the measured value

## Input example:

```
distance: 40  distance_uncertainty: 0.05
angle: 30 angle_uncertainty: 2

distance: 80  distance_uncertainty: 0.01
angle: 15 angle_uncertainty: 2

distance: 30  distance_uncertainty: 0.05
angle: 45 angle_uncertainty: 4
```

## Output example:

```
[32.684, 37.799]
[18.285, 24.703]
[24.775, 36.237]
```

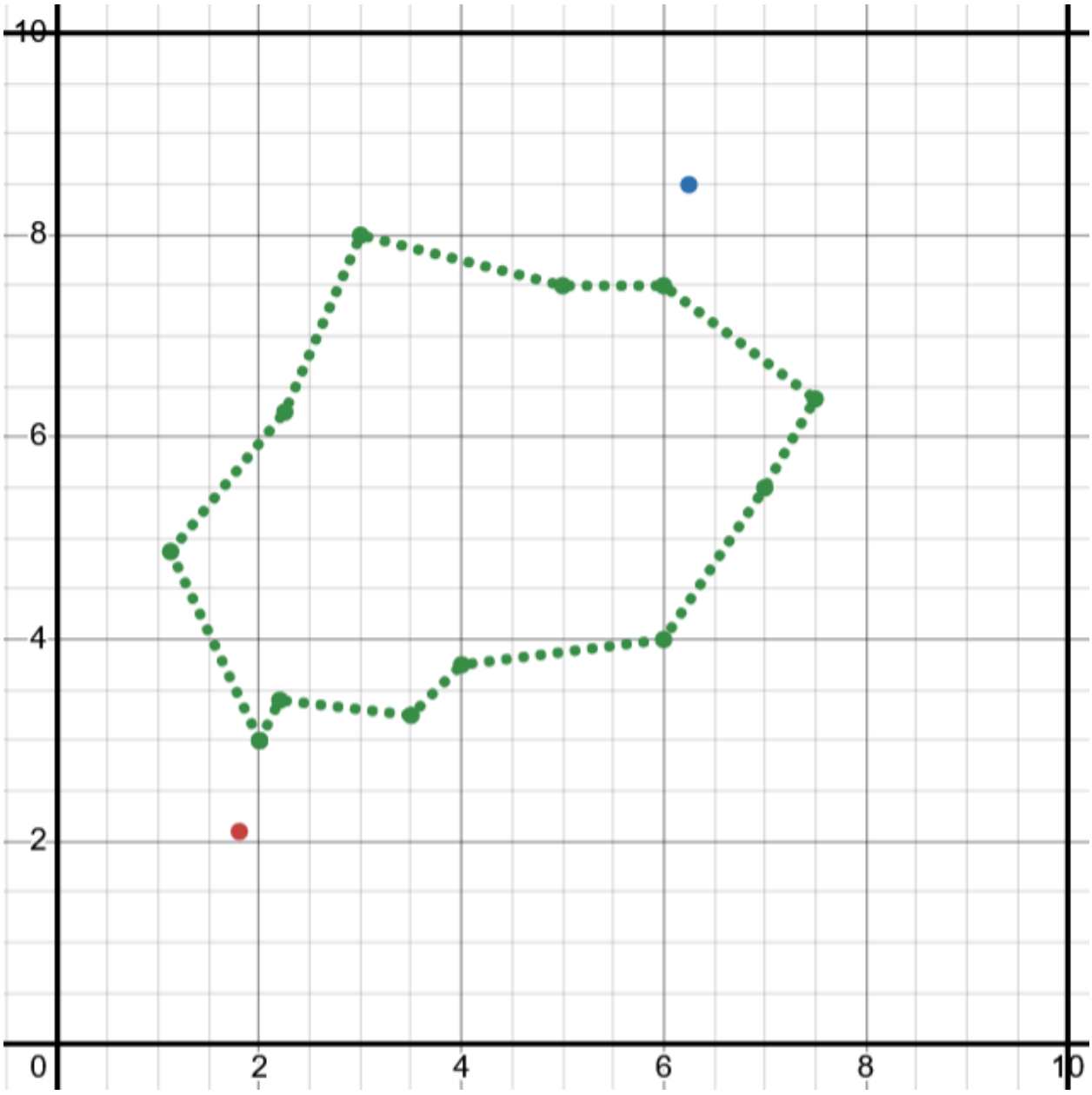
## Output explanation:

For the first example, we start by calculating the lower limit of the statue's size. To do this, we assume that the angle is at its smallest, and that the distance is also the smallest. We calculate the minimum distance as 40m * (1 - 0.05), which gives us 38 metres. For the angle, we take 30 - 2, i.e. 28 degrees. We can now calculate the hypotenuse using the formula given in the statement, which gives us 43.0376619262 metres. We can apply the Pythagorean theorem and find a height of 32.6839462745 metres. We then round up to 32.684 metres.

We apply the same calculations with a maximum distance of 42 metres and 32 degrees. This gives us a hypotenuse of 49.5254929412 metres and a height of 37.7991329407, which can be rounded up to 37.799 metres.

## WW7: Lighthouse of Alexandria (85 points)

A storm is brewing on the horizon and a fishing boat is trying to make it back to shore safely. The great Lighthouse of Alexandria can guide them, as long as its light is not completely covered by the great dark clouds that cover the sky. The problem will be solved on a Cartesian plane measuring 10km by 10km. Lighthouse and boat coordinates will be specified for each situation. A single series of points representing a cloud will also be given as input. A typical situation might look something like this:



The cloud is green (we've connected the dots to make it easier to see), the Lighthouse is red and the boat is blue. The lighthouse emits light in all directions, but obviously the **path of light reaching the boat is a straight line from the lighthouse to the boat**.

This is where we come up against the problem with clouds: **the intensity of the light coming from the Lighthouse will decrease by 20% for every km traveled through the cloud, and this will happen linearly** (i.e. 2 km traveled will decrease the intensity of the light by 40%). It's useful to be familiar with these mathematical concepts:

- Two points (x, y) are enough to determine the equation of the line connecting the two points, by determining the slope and the y-intercept.
- The point of intersection of two lines can be determined by putting two equal line equations together and solving on both sides.
- The distance between two points (x1, y1) and (x2, y2) is calculated using the following formula:

$$d = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

**If the intensity of the light reaching the boat drops to 0%, the fishermen on the boat won't be able to see it. So you'll need to decide whether or not the fishermen will be guided to shore by the lighthouse. The light enters the cloud exactly once and leaves it exactly once.**

## Input specifications:

As input you will receive:

- ***lighthouse***: a list containing two floats, i.e. lighthouse coordinates
- ***boat***: a list containing two floats, i.e. the coordinates of the boat
- ***cloud: a list of coordinates describing the shape of the cloud (i.e. a 2D array of floats)***

## Output specifications:

On exit, you must provide:

- a string displaying "OUI" if the fishermen are safe or "NON" if they will have to manage without the lighthouse

## Input example:

**`lighthouse:`**`[1.8, 2.1]`  **`boat:`**`[6.25, 8.5]`

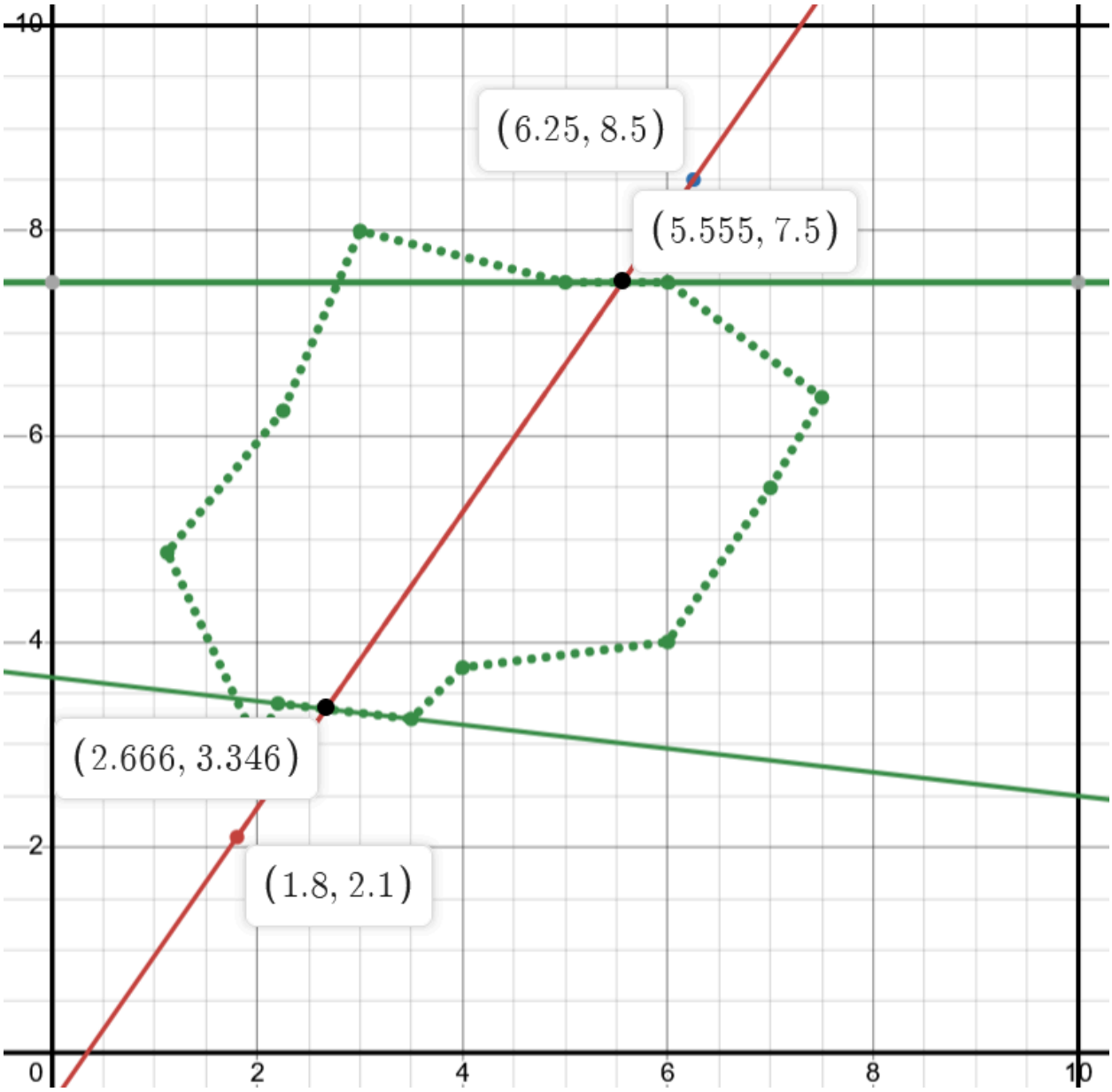
**cloud:**`[[2,3],[2.2,3.4],[3.5,3.25],[4,3.75],[6,4],[7,5.5],[7.5,6.38],[6,7.5],[5,7.5],[3,8],[2.25,6.25],[1.12,4.87]]`

## Output example:

`"NON"`

## Output explanation:

Calculating the line intersections where the light enters the cloud, we obtain the points (5.555, 7.5) and (2.666, 3.346). The distance between the two points is around 5.06 km, which drops the light intensity to 0% before it reaches the boat. So, the fishermen don't perceive the light and the answer is "NON". (see image below)




**Footnote:**

Light absorption is actually exponential, but it was simpler here to treat it linearly.

# MM Mastermind

## MM: Master of the mind (100 points)

The mastermind game is a strategy game in which you wish to discover the combination established by the mastermind. The combination to be guessed is a long 4 combination chosen from 6 symbols. The same symbol may appear more than once in the combination.

To find the right combination, you can make a few attempts, each of which will give you information on the validity of your guess. In return, you'll receive two types of information. One number will tell you how many symbols are in the right place, and another number will tell you how many symbols in your trial are valid but in the wrong place.

If you make a trial with 1 perfectly placed symbol and all the other symbols in the wrong place, you'll receive (1, 0) in return. If you try a combination with 3 valid but misplaced symbols, you'll receive the return (0, 3). With 1 perfect and two misplaced symbols you'll receive (1, 2). If you've won, you'll receive the return (4, 0). The symbols used for our version of the game are: / # = + - *

You'll have a total of 20 tries to find the right combination!

### Input specification:

As input, you will receive:

- the verification function to call to receive validation hints. You'll need to send your tests to this function, using the feedback to improve your next tests. This function takes as input an array of strings (the 4 symbols of your test)

### Output specification:

On exit, you must return the winning combination in an array with the four symbols of the combination in the correct order.

### The entry:

```
verif_function
```

### Output:

See the explanation…

### Explanation of the output:

We start our first test with the combination: - - # # and receive (0, 1) We now know that only one of the two symbols is valid and that it appears once in a place where it isn't currently. The 2nd try is: + = = - which returns (1,1) We now know that one of the symbols we tried is the right one in

the right place and that another symbol is valid, but misplaced. We continue the next trial with: / + / - which gives us the return: (1, 1) Exactly like the return of the previous combination. Our 4th trial is = = + - which gives us the return (0, 2) We now have 2 valid but misplaced tokens. We continue with the trial: # = / + which gives us (1, 3) We know we've almost reached the solution because all the chips are valid and one of them is in the right place. We finally try the combination / # = + which returns (4, 0). So we've found the right solution! Here's a table summarizing the trials and results.
The correct combination in this case was / # = +

| s0 | s1 | s2 | s3 | résultat |
|---|---|---|---|---|
| - | - | # | # | (0, 1) |
| + | = | = | - | (1, 1) |
| / | + | / | - | (1, 1) |
| = | = | + | - | (0, 2) |
| # | = | / | + | (1, 3) |
| / | # | = | + | (4, 0) |